

Guida Completa Flask API: Ambiente di Sviluppo, ORM e Swagger UI

Docente: **Moussa Bakabi Salisou**

Questo manuale didattico copre l'intero ciclo di vita dello sviluppo di un'API RESTful professionale utilizzando Python e **Flask**. Partendo da zero, vedremo l'organizzazione del workspace, la configurazione dell'ambiente virtuale protetto, l'integrazione di un database relazionale tramite **Flask-SQLAlchemy** e l'esposizione interattiva del software con **Swagger UI** sul dominio logico **ScuolaDB**.

Modulo 1: Inizializzazione del Workspace e Ambiente Virtuale

Isolare le dipendenze tramite un ambiente virtuale (`.venv`) garantisce la portabilità del codice ed evita conflitti tra librerie a livello di sistema operativo.

1.1 Creazione della Cartella e Setup dell'Ambiente Virtuale

Eseguire i seguenti comandi in sequenza all'interno del terminale di sistema:

```
# 1. Creazione della cartella principale del progetto
mkdir corso_flask_api
cd corso_flask_api

# 2. Creazione dell'ambiente virtuale isolato denominato .venv
python -m venv .venv
```

1.2 Attivazione del Virtual Environment

A seconda del sistema operativo in uso nella postazione di laboratorio, eseguire il comando specifico:

- **Windows (Prompt dei comandi - cmd):** `.venv\Scripts\activate.bat`
- **Windows (PowerShell):** `.venv\Scripts\Activate.ps1`
- **Linux / macOS:** `source .venv/bin/activate`

Nota di verifica: L'avvenuta attivazione è confermata dalla comparsa del prefisso `(.venv)` prima del percorso nel terminale.

1.3 Installazione Completa dei Pacchetti Professionali

Con l'ambiente virtuale attivo, aggiornare il gestore pacchetti ed installare i componenti necessari:

```
python -m pip install --upgrade pip
pip install flask flask-sqlalchemy flask-restx
```

Modulo 2: Architettura Logica delle Risorse ed Endpoint CRUD

L'API mapperà le operazioni fondamentali di gestione sulla tabella `studenti` del database relazionale locale:

Metodo HTTP	Rotta (Endpoint)	Azione CRUD	Stato HTTP di Successo
GET	<code>/api/studenti</code>	Read (Ottiene tutti gli studenti)	200 OK
POST	<code>/api/studenti</code>	Create (Aggiunge un nuovo studente)	201 Created
GET	<code>/api/studenti/<id></code>	Read One (Dettaglio del singolo studente)	200 OK
PUT	<code>/api/studenti/<id></code>	Update (Modifica completa dello studente)	200 OK
DELETE	<code>/api/studenti/<id></code>	Delete (Rimozione record dal database)	200 OK

Modulo 3: Implementazione del Codice Sorgente Unificato (app.py)

Creare un file denominato `app.py` nella radice del progetto e salvare il seguente script di produzione didattica:

```

from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_restx import Api, Resource, fields

app = Flask(__name__)

# --- CONFIGURAZIONE STRATO DI PERSISTENZA REALE (SQLite) ---
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///scuola.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

# --- INIZIALIZZAZIONE SPECIFICHE OPENAPI / SWAGGER UI ---
api = Api(app,
          version='1.0',
          title='ScuolaDB API RESTful',
          description='Infrastruttura per la gestione automatizzata degli studenti tramite Swagger UI',
          doc='/swagger')

# Definizione del Namespace per organizzare la documentazione
ns_studenti = api.namespace('api/studenti', description='Operazioni CRUD complete')

# --- MODELLO STRUTTURALE DATABASE (ORM TABLE) ---
class StudenteModel(db.Model):
    __tablename__ = 'studenti'
    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    nome = db.Column(db.String(50), nullable=False)
    cognome = db.Column(db.String(50), nullable=False)
    classe = db.Column(db.String(10), nullable=True)

# --- MODELLI DI DATA VALIDATION & TRASFORMAZIONE JSON (SWAGGER SCHEMAS) ---
studente_input_schema = api.model('StudenteInput', {
    'nome': fields.String(required=True, description='Nome dell\'alunno'),
    'cognome': fields.String(required=True, description='Cognome dell\'alunno'),
    'classe': fields.String(description='Classe di appartenenza (es. 5A)')
})

studente_output_schema = api.inherit('StudenteOutput', studente_input_schema, {
    'id': fields.Integer(description='Identificativo univoco generato dal DB relazionale')
})

# --- CONTROLLER / RISORSE LOGICHE HTTP ---

@ns_studenti.route('')
class ListaStudenti(Resource):

    @ns_studenti.marshal_list_with(studente_output_schema)
    def get(self):
        return StudenteModel.query.all(), 200

    @ns_studenti.expect(studente_input_schema)
    @ns_studenti.marshal_with(studente_output_schema, code=201)
    def post(self):
        payload = api.payload
        nuovo_studente = StudenteModel(
            nome=payload['nome'],
            cognome=payload['cognome'],
            classe=payload.get('classe', 'N/D')
        )
        db.session.add(nuovo_studente)
        db.session.commit()

```

```

        return nuovo_studente, 201

@ns_studenti.route('/<int:id>')
@ns_studenti.response(404, 'Risorsa studente non localizzata nel database')
class DettaglioStudente(Resource):

    @ns_studenti.marshal_with(studente_output_schema)
    def get(self, id):
        studente = db.session.get(StudenteModel, id)
        if not studente:
            return {"errore": "Record non trovato"}, 404
        return studente, 200

    @ns_studenti.expect(studente_input_schema)
    @ns_studenti.marshal_with(studente_output_schema)
    def put(self, id):
        studente = db.session.get(StudenteModel, id)
        if not studente:
            return {"errore": "Record non modificabile perché inesistente"}, 404
        payload = api.payload

        studente.nome = payload['nome']
        studente.cognome = payload['cognome']
        studente.classe = payload.get('classe', studente.classe)

        db.session.commit()
        return studente, 200

    def delete(self, id):
        studente = db.session.get(StudenteModel, id)
        if not studente:
            return {"errore": "Impossibile eliminare: record inesistente"}, 404
        db.session.delete(studente)
        db.session.commit()
        return {"messaggio": "Record rimosso con successo"}, 200

# --- CREAZIONE CONTESTUALE AUTOMATICA DEL DATABASE ---
with app.app_context():
    db.create_all()

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

Modulo 4: Avvio e Interazione Interattiva con Swagger UI

Eseguire l'applicazione lanciando l'interprete Python sul file principale:

```
python app.py
```

4.1 Controllo Visivo di Swagger nel Browser

Aprire un browser web moderno e digitare l'indirizzo della documentazione: `http://127.0.0.1:5000/swagger`. Verrà mostrata la console interattiva:

```
-----  
📄 ScuolaDB API RESTful v1.0 [Swagger Console]  
-----
```

```
📁 api/studenti : Operazioni CRUD complete
```

```
[ GET ]    /api/studenti -----> Elenco completo degli studenti nel DB  
[ POST ]   /api/studenti -----> Crea e valida un record studente  
  
[ GET ]    /api/studenti/{id} -> Trova record per ID univoco  
[ PUT ]    /api/studenti/{id} -> Modifica dati totali per ID  
[ DELETE ] /api/studenti/{id} -> Rimuove la riga dal database  
-----
```

4.2 Esecuzione dei Test CRUD di Laboratorio

1. **Test Inserimento (POST):** Espandere la riga `POST`, cliccare su **Try it out**, inserire i valori per nome e cognome nel corpo JSON generato automaticamente e cliccare su **Execute** per visualizzare la risposta `201 Created`.
2. **Test Lettura (GET All):** Espandere la rotta `GET /api/studenti`, cliccare su **Try it out** e premere **Execute** per riscontrare la lista contenente i record precedentemente inseriti nel file reale `instance/scuola.db`.

💡 Esercizio di Consolidamento per la Classe

Invitare gli studenti a verificare la persistenza spegnendo il server tramite la combinazione di tasti `CTRL + C` nel terminale, riavviandolo successivamente con `python app.py` ed eseguendo una chiamata `GET` per constatare che i record inseriti rimangono memorizzati stabilmente.